CURRENT IMPLEMENTATION, IN PLAIN ENGLISH

One conversation. A supervised fleet.

Firstmate is a durable operating system for agent work: it turns a captain's request into isolated tasks, keeps watch without spending model tokens, preserves enough truth to recover after interruption, and never treats a notification as proof that the work is safe.

Human-facing crew **Mechanical IDs underneath** **Event-driven supervision**

Explicit merge authority

ORIENTATION

Evidence-linked

Read this first

The page separates what Firstmate does now, what is a known limit, and what is only a possible design direction.

Legend **Current behaviour** **Verified limitation** **Proposed enhancement**

⚓ **Names have two layers.** HMS *Surprise* and Pullings are display identities for the captain-facing experience, but scripts still address work through stable mechanical task identifiers such as `feature-login` and endpoint labels derived from them. The friendly name can change without moving the task's branch, worktree, status ledger, or backend endpoint.

This explanation intentionally omits live home contents, private paths, credentials, tokens, and message bodies. File links point only to tracked implementation and documentation.

Product overview **Maintainer architecture** **Validation evidence**

01

Current behaviour

The mental model

The captain talks to one primary Firstmate. That Firstmate routes work to persistent domain leads or temporary specialists, each with an isolated place to work.

C

Captain

Sets intent, answers material choices, reviews outcomes, and retains merge authority unless an explicit standing posture allows routine green merges.

1M

Primary Firstmate

Owns intake, routing, supervision, escalation, durable fleet records, and the final plain-English report back to the captain.

2M

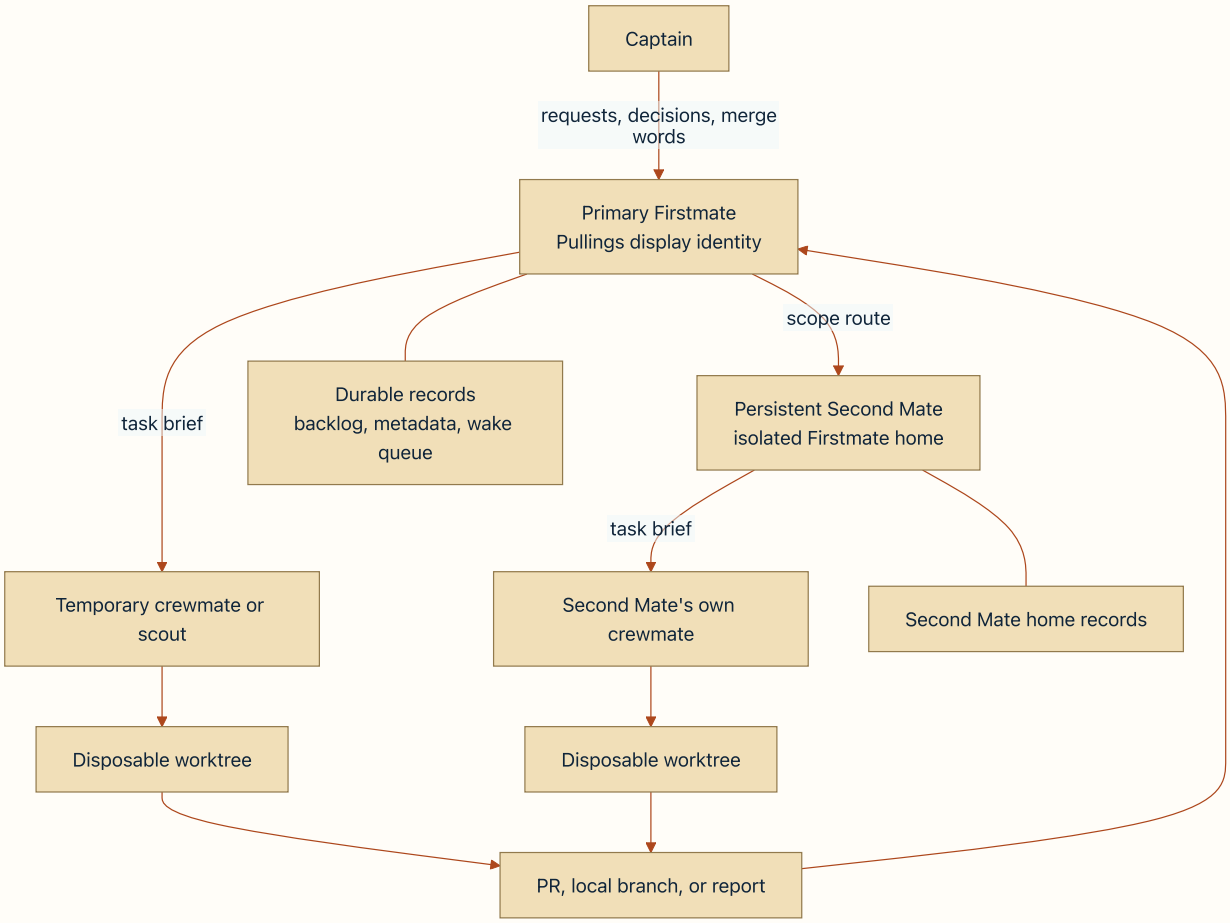
Second Mate

A persistent Firstmate in an isolated home with a charter and scope. It can supervise its own crewmates and waits silently when its queue is empty.

W

Crewmate or scout

A temporary autonomous worker in a disposable worktree. A crewmate ships change; a scout produces a standalone report and never pushes.



At a glance: humans see a crew hierarchy, while every worker still resolves to a mechanical task record, endpoint, and isolated copy.

Project clone

The home's long-lived, guarded local clone used for orientation, registry operations, and safe refreshes. The primary Firstmate normally reads it rather than implementing inside it.

Task worktree

A separate working copy for one task. Ship and scout spawns refuse to start if the resolved path is the primary project checkout instead of a genuine isolated worktree.

▼ Technical evidence for the mental model

The product overview defines the one-liaison model, visible workers, disposable worktrees, two task shapes, and optional Second Mates. The spawn script and its isolation regression enforce the distinct-worktree boundary rather than leaving it as prose.

README | Worktree architecture | Spawn owner | Isolation regression

02

Current behaviour

The component map

Firstmate is not a single daemon or model. It is a tracked agent distribution plus private operational homes, helper scripts, harness integrations, and a runtime backend.

Tracked distribution

- `AGENTS.md` gives the always-loaded operating contract.
- `.agents/skills/` holds conditional procedures.
- `bin/` owns mechanical actions and safety checks.
- `docs/` explains stable behaviour and evidence.
- Harness hook directories adapt lifecycle events to shared scripts.

Private operational home

- `data/` holds backlog, preferences, registries, and reports.
- `state/` holds task identity, notifications, locks, wakes, and recovery records.
- `config/` holds home-local choices such as backend and harness routing.
- `projects/` holds guarded project clones.
- `.env` is optional, private, and never part of tracked documentation.

Four terms that sound similar but are not

LAYER	PLAIN-ENGLISH MEANING	EXAMPLES	WHAT IT DOES NOT DECIDE
Harness	The agent program that reads the brief, calls tools, and presents a conversation.	Codex, Claude, Pi, Grok, OpenCode, Cursor, Kimi, Muse.	It is not the language model and it is not the terminal placement layer.
Model	The reasoning model selected inside or through the harness.	A configured model name recorded with a spawn profile.	It does not own the worktree, terminal, or delivery mode.
Runtime backend	The session provider that creates, finds, sends to, and observes the worker endpoint.	tmux, Herdr, Zellij, Orca, cmux.	It does not choose the model or interpret the task.
Herdr Space	An optional disposable visual workspace used when the runtime backend is Herdr.	A projected one-task workspace under the launching home's workspace.	It never becomes task, endpoint, send, or cleanup authority.

> **A practical example:** a worker can run the Codex harness, use a selected model, live in a Herdr runtime endpoint, and appear in a disposable Herdr Space. These are four separate facts recorded or resolved by different owners.

▼ **Key script families and their responsibilities**

Start and recover

`fm-session-start`, `fm-bootstrap`, `fm-startup-network`, and `fm-wake-drain` establish authority and present the work queue.

Create and communicate

`fm-brief`, `fm-spawn`, `fm-send`, and `fm-control` create instructions, start work, send text, and operate lifecycle controls.

Watch and reconcile

`fm-watch`, `fm-crew-state`, and `fm-classify-lib` detect events, distinguish history from current state, and route actionable wakes.

Deliver

`fm-pr-check`, `fm-pr-merge`, `fm-merge-local`, and no-mistakes adapters preserve delivery and approval boundaries.

Clean up

`fm-teardown` owns the landed-work proof and refuses dirty or unlanded work unless explicit discard authority exists.

Persistent domains

`fm-home-seed`, registry helpers, remote controls, and backlog handoff create and maintain Second Mate homes.

Toolbelt index **Harness support** **Backend selection** **Herdr Spaces**

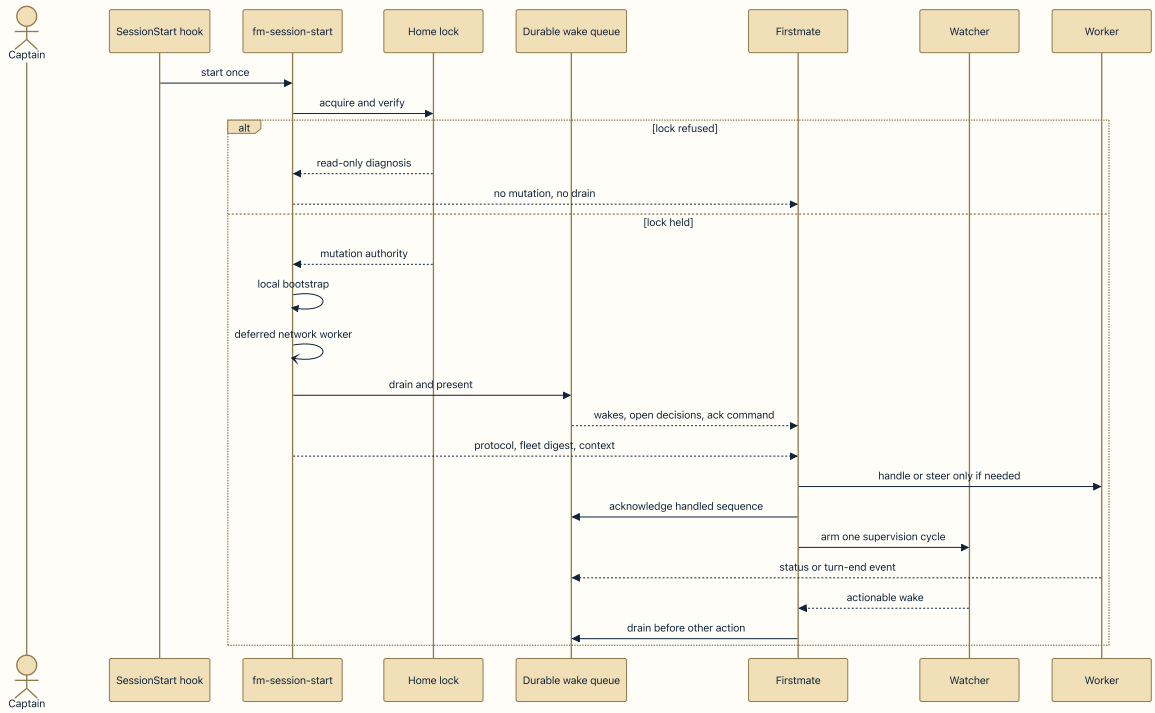
03

Current behaviour

Session start and the supervision loop

A session first proves it may act, then presents durable work, then keeps one supervision cycle alive until the fleet is quiet.

1. **Acquire the home lock.** A session that cannot prove lock ownership stays read-only.
2. **Run local bootstrap checks.** Mutating sweeps run only for the lock owner.
3. **Start network checks off the blocking path.** Authentication, remote routes, handoffs, and clone refreshes can finish later and report durably.
4. **Present the wake queue and open decisions.** Presented rows remain until the exact acknowledgement is issued after handling.
5. **Emit one harness-specific supervision protocol.** The script does not invent a generic wait shape.
6. **Print fleet and context digests.** Live task identity comes before curated context so truncation loses the cheaper information last.
7. **Return to the watch.** Exactly one cycle waits for actionable change and wakes the primary Firstmate.



The durable queue is the hand-off between shell supervision and model reasoning. A wake is not consumed merely because it was printed.

Heartbeat

A bounded prompt to review the whole fleet when normal events have been quiet. No change is an expected result, not progress to report.

Wake

A durable queue record saying something may require Firstmate attention. It carries a sequence and survives interruption until acknowledged.

Stale

An endpoint or worker has remained unchanged beyond a safety cadence. It is a demand to inspect, not permission to interrupt or restart automatically.

Open decision

A keyed `needs-decision` or blocker event that remains visible until the matching resolution is appended or transferred into durable captain-owned work.

! **Why Pullings can be prevented from ending a turn:** if work, a registered event source, or Relay polling still needs supervision and no healthy identity-matched watch is proven, the stop or turn-end integration blocks the stop or schedules one bounded follow-up. This prevents a quiet-looking turn ending with nobody watching the fleet. The mechanism is bounded per harness so a broken guard does not trap the session forever.

▼ How the watcher decides what is actionable

The shell watcher absorbs routine no-change heartbeats and positively proven busy work. It surfaces captain-relevant status, unproven no-verb signals, authenticated check results, overdue declared waits, stale or wedged workers, and heartbeat reviews. It appends actionable records to `state/.wake-queue` before waking the primary session.

Status files are append-only event logs, so the drain separately folds keyed open decisions and unread notes. This prevents an unresolved decision from disappearing beneath a later progress line.

Supervision architecture **Watcher** **Wake drain** **Status classification** **Queue regression**

▼ How session start is kept bounded

The blocking digest performs local work only. GitHub authentication, Second Mate convergence and liveness, pending remote handoff delivery, and project clone refresh run in a separately bounded deferred stage. If they are still running, the digest says what remains unconfirmed instead of claiming success.

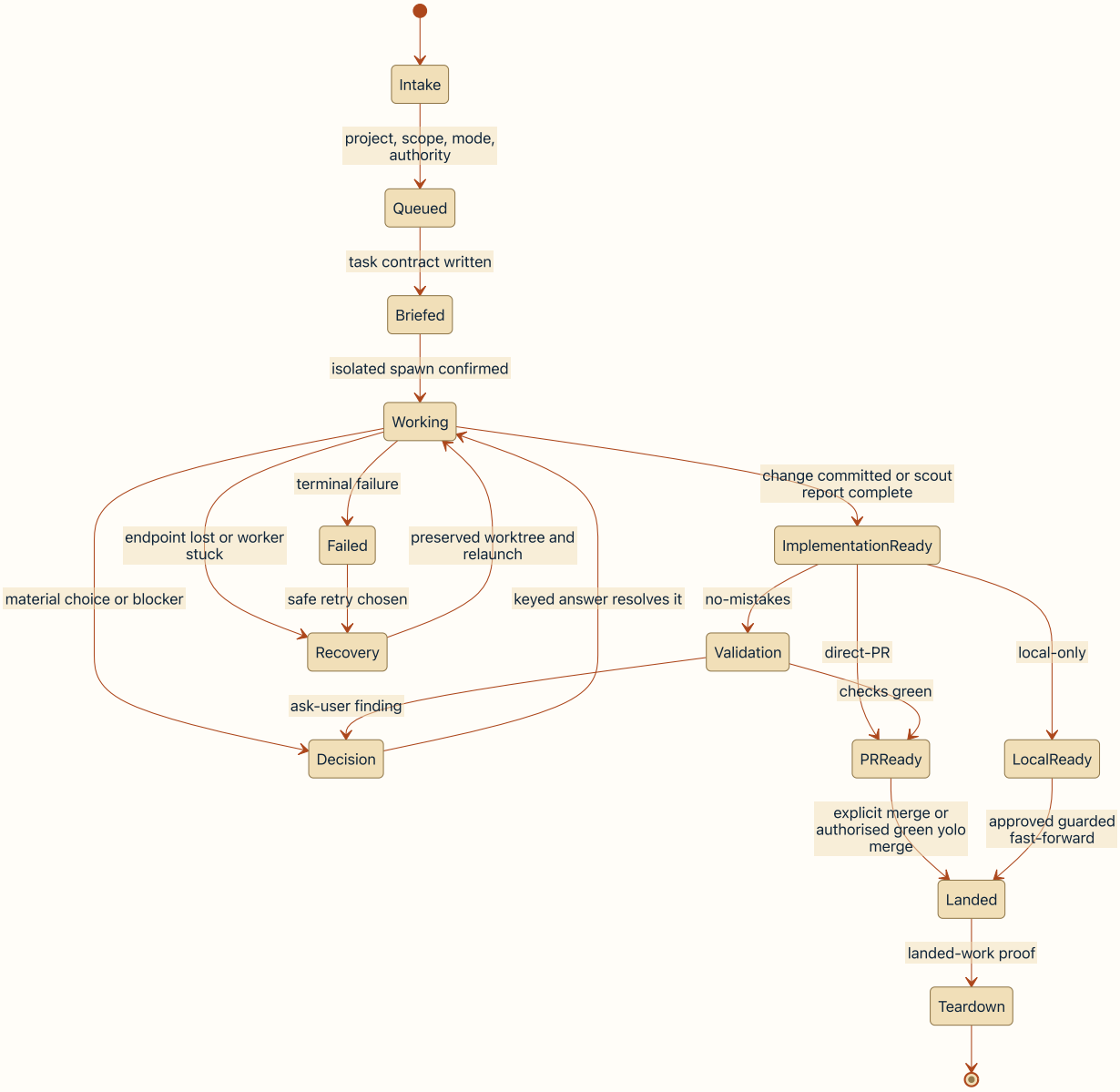
Session-start composition **Deferred network stage** **Session-start regression** **Network-stage regression**

04

Current behaviour

The task lifecycle

The lifecycle carries the same intent through intake, isolation, work, decisions, validation, landing, and safe cleanup.



This is the operational lifecycle, not a list of backlog enum values. A worker's `done:` signal can mean implementation-ready while Firstmate still waits for landing before cleanup.

Three delivery modes

MODE	WORKER DELIVERY	READY SIGNAL	LANDING AUTHORITY
no-mistakes	The pipeline owns review, fixes, tests, docs, lint, push, PR, and CI.	PR URL with checks green.	Captain with yolo off; Firstmate may merge routine green work with valid yolo posture.
direct-PR	The worker performs proportionate checks, pushes, and opens a PR without the no-mistakes pipeline.	PR URL after opening.	The same explicit merge boundary applies.
local-only	The worker leaves a clean ready branch without pushing.	Clean local branch ready to land.	Firstmate uses the guarded fast-forward path only after approval; this work stays in the main home.

✓ **Delivery mode is not merge authority.** Mode selects the route and rigour. The independent `yolo` posture controls routine gate and green-merge autonomy within accepted intent. Destructive, irreversible, security-sensitive, expanded-scope, and red-PR choices retain stronger captain boundaries.

Intake

Resolve the project, route by Second Mate scope, decide ship versus scout, choose concrete delivery mode and yolo posture, and write a task-specific brief.

Execution

Spawn into a fresh isolated worktree, verify the worker is processing the brief, supervise sparse status events, and answer keyed decisions through the data plane.

Landing

Use the selected validation path, record the PR or local result, wait for authority, land through guarded helpers, then prove the work is landed before teardown.

▼ Text messages, lifecycle control, and teardown are separate

`fm-send` is the data plane for text an agent should read. `fm-control` is the control plane for the closed verbs `interrupt`, `exit`, and `relaunch`, each with a backend-verifiable postcondition. Teardown is deliberately outside that control plane because it can remove an endpoint and worktree.

Control-plane contract **Data plane** **Control plane** **Teardown owner**

▼ Decision and validation ownership

A worker reports a material product or engineering choice as a keyed decision. Firstmate applies the configured authority, passes the exact answer back with the same key, and the resolution append closes that open record. During no-mistakes validation, the pipeline alone owns fixes and branch custody; the implementation worker drives every gate until checks pass or a genuine captain choice is escalated.

Durable decisions **Decision owner** **Gate authority** **PR readiness record**

05

Current behaviour

The Second Mate lifecycle

A Second Mate is persistent organisational capacity, not a long-running ordinary task and not a separate architecture.

What makes it persistent

- Its own `FM_HOME`, data, state, config, projects, and session lock.
- A charter stored in its home and a route recorded in the primary home's Second Mate registry.
- A natural-language scope used for routing; project lists help provision the home but do not grant exclusive ownership.
- Its own crewmates, endpoints, backlog, decisions, and recovery loop.

What keeps it bounded

- It reconciles existing work after restart, then idles if no task has been routed.
- It never invents audits, surveys, or improvements from an empty queue.
- The primary reads routed replies from status or a referenced document, not by scraping the Second Mate's chat.
- Retirement is explicit and refuses while child work remains in flight.

Lifecycle in six steps

1. **Seed:** create an isolated home, projects or project-less lease, charter, and registry entry transactionally.
2. **Launch:** resolve its harness pin and backend, then start it as a direct report with its own home.
3. **Route:** move judged in-scope backlog work into that home's queue and send a marked request.
4. **Supervise:** the Second Mate runs its own session-start and watcher loop and can dispatch its own isolated workers.
5. **Recover:** a confirmed dead or missing agent can be relaunched against the preserved home; an unreachable remote route remains unknown and is never silently replaced locally.
6. **Retire:** only an explicit decision can remove the persistent route, and the home must have no work under way unless discard is separately authorised.

↔ **The captain can still talk directly to a Second Mate.** A marked Firstmate request is routed back through status and correlation records. Direct human typing stays unmarked and conversational, and Firstmate reconciles that authoritative intervention at the next supervision review.

▼ Inherited configuration and remote homes

The primary conservatively propagates a declared allowlist of local configuration and shared captain preferences. The primary's `secondmate-harness` selection is not inherited as a child preference because Second Mates do not create more Second Mates. A remote Second Mate places the whole home on an SSH-reachable host, pins that agent to Herdr, and keeps its own workers' backend selection independent.

Second Mate routes **Remote homes** **Home seeding** **Backlog handoff** **Lifecycle regression**

06

Current behaviour

Hooks and safety controls

Hooks enforce narrow, mechanical boundaries. They do not replace the reasoning, authority, and reporting responsibilities carried by the operating contract.

CONTROL	WHAT IT CAN ENFORCE	WHAT REMAINS A MODEL OR PROCESS RESPONSIBILITY
SessionStart	Invoke the bounded start owner or nudge the correct start path for the harness.	Understand the digest, reconcile decisions, and act only with verified lock authority.
PreToolUse seatbelts	Block unsafe watcher-arm command shapes, persistent top-level directory changes, and supported subagent creation surfaces in primary scope.	Choose the right project, route work, avoid dangerous commands outside covered shapes, and respect captain scope.
Stop or turn-end guard	Detect supervision need and either block a stop or issue one bounded follow-up through the harness's verified mechanism.	Handle the wake, repair continuity, and report only captain-relevant outcomes.
Watcher protection	Use identity-matched locks, fresh beacons, durable queue ordering, and bounded recovery acknowledgements.	Interpret a stale event and decide whether to inspect, steer, relaunch, or escalate.
Project boundary	Refuse unsafe spawn locations, restrict Firstmate's primary writes, and route project changes through isolated workers.	Keep the task within accepted intent and avoid changing unrelated user work.
Landing and teardown guards	Require explicit merge paths, reject red or malformed PR operations, and refuse dirty or unlanded worktree removal.	Obtain the correct merge or discard authority and explain failures faithfully.

No unlanded work

Teardown is a proof, not tidying. A dirty worktree or committed change that is not demonstrably landed stops cleanup for investigation.

Explicit merge authority

No PR merge occurs from implication. The captain's concrete instruction or a valid yolo posture for routine green work is required, and every task merge uses the guarded helper.

Secret boundaries

Private home state and optional tokens live in gitignored surfaces. Tracked docs, briefs, reports, and public replies must not copy credentials or sensitive message bodies.

▼ Tracked hook integrations

Claude, Codex, Cursor, Grok, Pi, and OpenCode use different host events and response mechanisms, but converge on shared scripts for session start, pre-tool checks, and turn-end safety where supported. Some host boundaries deliberately step aside when prerequisites are unreadable because blocking every command or trapping a session would be worse than losing an optional check.

Claude hooks **Codex hooks** **Cursor hooks** **Grok hooks** **Pi extension** **OpenCode plugin**

▼ Safety evidence

Turn-end guard contract **Watcher-arm seatbelt** **Directory-change seatbelt** **Subagent boundary** **Turn-end regression** **Directory guard regression**

07

Current behaviour

Durable state and recovery

Firstmate recovers from disk and live endpoint facts, not from confidence, chat scrollback, or the latest status sentence.

Effective Firstmate home/	The operational domain selected by `FM_HOME`, which falls back to the tracked code root when unset.	HOME ROOT
data/	Backlog, project and Second Mate registries, preferences, charters, and scout reports.	DURABLE INTENT
state/	Runtime identity, event history, locks, queues, checks, and recovery records.	DURABLE OPERATIONS
<id>.meta	Task kind, mode, harness, backend, branch, worktree, endpoint, and related identity.	TARGET IDENTITY
<id>.status	Append-only wake events such as working, blocked, needs-decision, resolved, done, and failed.	HISTORY ONLY
.wake-queue	Sequenced actionable notifications awaiting handling and acknowledgement.	DURABLE HAND-OFF
.lock	The verified harness session that currently owns this home's mutation authority.	SESSION AUTHORITY
.last-watcher-beat	Freshness beacon touched only by the watcher.	LIVENESS EVIDENCE
decision-bindings/	Bindings that let keyed captured answers close the correct durable captain decision.	DECISION ROUTING

`config/`	Backend, harness, dispatch, calm, inherited material, and local operational choices.	HOME CONFIGURATION
`projects/`	Guarded long-lived clones used for routing and safe project operations.	PROJECT BASELINE
`task worktree`	The isolated working copy whose Git state is the authority for uncommitted and committed task changes.	WORK PRODUCT

Important hierarchy: a status line can wake Firstmate, but it does not override the worktree, metadata identity, live endpoint, current no-mistakes run, backlog, or a registered Second Mate home's own structured state.

How current state is reconstructed

1. Read the task metadata to identify the exact branch, worktree, harness, backend, and endpoint.
2. Attribute a current no-mistakes run only when its branch and code identity match the task.
3. Otherwise ask the backend for semantic busy, idle, dead, or unknown evidence.
4. Only a proven idle endpoint may fall back to a recognised status event; unknown never becomes working or done by guesswork.
5. For registered Second Mates, their own validated home outranks the parent task's historical status lines.

OBSERVED CONDITION	WHAT FIRSTMATE TRUSTS	SAFE RESPONSE
Dead endpoint	Exact metadata plus backend agent-state classification and preserved worktree.	Load recovery procedure and relaunch in place without discarding work.
Stale worker	Current state, live pane evidence, turn-end age, and validation logs where relevant.	Inspect deeply; never infer that stale means dead or auto-interrupt.
Pending Second Mate reply	Parent-owned correlation and retry record, not the Second Mate's chat transcript.	Retry or escalate through the durable pending-reply path.
Unresolved decision	Keyed open-decision fold or structured captain-owned backlog item.	Present the concrete choice and close it only with a matching answer.
Dirty worktree	Git status and landed-work proof.	Refuse teardown and preserve the copy for investigation.
Watcher loss	Identity-matched lock, process identity, fresh beacon, recovery generation, and durable queue.	Restore one cycle, re-present unacknowledged work, and use the printed generation-bound acknowledgement.

▼ Why chat memory is deliberately secondary

A conversation can be compacted, a terminal can disappear, and a status event can become stale. Briefs, worktrees, metadata, backlogs, queue rows, decision keys, checks, and live backend observations survive those failures and can be reconciled by a fresh session. Terminal scrollback is useful for diagnosis but never overrides valid structured state.

Restart-proof architecture **Current-state reconciler** **Current-state regression** **Watcher continuity**

08

Current plus limits

What the captain sees

The current experience is intentionally transparent: visible worker surfaces plus concise Firstmate outcomes. Presentation cleanliness depends on the harness and backend.

Current behaviour

Spaces and crew labels

Each worker appears in a backend-specific endpoint such as a tmux window, Herdr tab or Space, Zellij tab, Orca terminal, or cmux workspace. Human-facing labels make the fleet navigable while durable metadata retains the mechanical identity.

Current behaviour

Pullings versus raw transcript

Pullings reports the outcome, consequence, decision, and full PR link in the primary conversation. The worker panes remain available for direct observation, including tool rows and operational messages that are not repeated into captain-facing summaries.

Current behaviour

Direct Second Mate conversation

The captain can type directly into a Second Mate's endpoint. That message is conversational rather than a routed Firstmate control message, and the next supervision review reconciles any resulting change.

Verified limitation

Codex has no supported in-pane tool-row filter

The tracked Codex integration supplies session-start, pre-tool, and stop hooks, but no supported transcript-rendering extension point for hiding its tool rows. Firstmate therefore cannot present a clean filtered Codex worker pane today.

π **Pi Calm is different and harness-specific.** Pi's tracked `/calm` extension can hide supported transcript chrome and operational rows, replace the working indicator, and preserve the underlying context and exports. It is not a general Firstmate display layer and does not imply that Codex or other harnesses can filter their panes.

A clean Captain View is only a possible enhancement

Captain View concept — Not implemented

Under way	Captain's call	Ready to review
Two outcomes described in captain language, without tool rows.	One concrete choice with evidence and consequences.	One green PR with a full link and risk summary.

◇ **Proposed enhancement:** a separate read model could present only structured fleet outcomes and captain decisions while leaving every underlying worker transcript intact. No such unified Captain View exists in the current implementation, and this concept does not grant a display layer any new task or lifecycle authority.

▼ **Presentation evidence and limits**

Visible crew · Herdr presentation · Pi Calm current behaviour · Cross-harness Calm evidence · Codex integration surface

09

Current behaviour

Worked scenarios

These examples show the difference between what happens internally and what the captain needs to hear.

01. Build a new feature

1. Resolve the project and delivery posture.
2. Route by Second Mate scope or keep it in the main home.
3. Create a precise brief and spawn an isolated worktree.
4. Supervise until implementation is

Captain sees

A concise feature outcome, the complete PR URL or local-ready result, risk where relevant, and any actual choice.

02. Answer a decision

1. The worker opens a privacy-safe decision key.
2. The wake drain keeps that choice visible even if later progress events arrive.
3. Pulling s presents evidence, consequence, options, and a recom

Captain sees

The real choice in plain language, not a status prefix, internal hold, or raw pipeline finding.

commi
tted.

5. Run
the
select
ed
deliver
y path
and
report
the
ready
result.

menda
tion.

4. The
answe
r is
sent
with
the
same
key,
which
appen
ds the
resolut
ion.
5. Any
authori
sed
follow-
up
remain
s
linked
in
durabl
e
backlo
g
state.

03. An agent crashes

1. A dead or stale signal wakes supervision.
2. Firstmate reconciles metadata, backend agent state, worktree Git state, and validation ownership.
3. If recovery is safe, it relaunches against the same preser

Captain sees

Nothing for an automatic safe recovery; a concrete blocker only if the work cannot continue without help.

04. The captain intervenes directly

1. The captain types into a worker or Second Mate endpoint.
2. That instruction is authoritative within its concrete scope.
3. At the next supervision review, Pulling s reconciles the live endpoint and

Captain sees

No duplicate lecture or transcript relay; only a material consequence that now needs attention.

ved worktree and brief plus a progress note.

4. If the endpoint is unknown, Firstmate stops short of guessing or destroying anything.

durable records.

4. Any conflict, new decision, or delivery impact is recorded and handled through the normal lifecycle.

05. Work is ready to merge

1. The PR path reports readiness at its mode-specific point.
2. Firstmate records the exact PR and arms merge observation.
3. The captain explicitly says to merge, or valid yolo authority covers

Captain sees

The complete PR URL before merge, then a one-line landed outcome after the merge succeeds.

routine green landing.

4. The guarded merge helper records the result.
5. Only confirmed landed work can be torn down.

▼ **Scenario evidence**

Delivery modes **Decision lifecycle** **Relaunch** **PR merge owner** **Local merge owner** **Cleanup proof**

10

Reference

Glossary

The short definitions used throughout this page.

Firstmate home	One isolated operational domain selected by `FM_HOME`, containing private data, state, config, projects, and a session lock.
Project clone	The home's long-lived guarded clone, distinct from each temporary task working copy.
Worktree	An isolated Git working copy for one ship or scout task.
Harness	The agent program that reads prompts, calls tools, and exposes lifecycle events.
Model	The reasoning model selected for an agent run.
Backend	The runtime session provider that creates and observes worker endpoints.
Herdr	An experimental runtime backend with tabs and optional disposable presentation Spaces.
Wake	A durable actionable notification queued for Firstmate handling.
Status event	One append-only worker report. It may trigger attention but is not current-state truth by itself.
Watcher	The zero-token shell process that observes fleet state and writes actionable wakes.
Decision key	A stable privacy-safe identifier that binds an unresolved choice to its eventual answer.
no-mistakes	The full validation and delivery pipeline for review, fixes, tests, documentation, lint, push, PR, and CI.

PR	A pull request used for review and landing in PR-based delivery modes.
Teardown	Guarded cleanup after a task is proven landed or, for a scout, its report and decision inventory are complete.
Yolo posture	Standing authority for Firstmate to decide routine gates and merge green work within already accepted intent.

11

Reference

Frequently asked questions

The implementation boundaries that are easiest to misunderstand.

▼ Is Firstmate a model, harness, app, skill, or MCP server?

No. It is an agent distribution made of instructions, skills, tooling, policies, and state conventions. A supported harness launched inside it instantiates the primary Firstmate.

▼ Why not trust the latest status line?

Because status is append-only notification history. A later progress line can sit above an unresolved decision, and a terminal-looking line can be older than a live validation run. Firstmate reconciles the current run, endpoint, worktree, and structured records instead.

▼ Does stale mean the agent is dead?

No. Stale means the endpoint has not changed within the expected cadence. A busy-but-wedged worker, an idle stopped worker, an external wait, and an unreadable endpoint need different responses.

▼ Can Firstmate merge without asking?

Only when the project's standing yolo posture authorises routine green landing within accepted scope. With yolo off, the captain must explicitly state the concrete merge. Red, destructive, irreversible, security-sensitive, or expanded-scope choices retain stronger boundaries.

▼ Does teardown discard my uncommitted work?

No. Dirty or unlanded ship work refuses teardown. Explicit discard authority is separate and cannot be inferred from a desire to clean up.

▼ Why does the stop guard sometimes keep Pullings working?

Because work or an event source still needs a verified supervisor and no healthy watch has been proven. The guard creates a bounded continuation opportunity so the session does not end blind.

▼ Can I speak directly to a Second Mate?

Yes. Direct typing stays conversational. Firstmate-routed messages use a separate marker and correlation path so replies return to the parent without scraping chat.

▼ Is Herdr required?

No. tmux is the reference default, with Herdr, Zellij, Orca, and cmux available under their documented support boundaries. A Herdr Space exists only on the Herdr path.

▼ Can Codex hide its tool rows like Pi Calm?

Not through a supported Firstmate in-pane filtering surface today. Pi Calm is implemented through Pi-specific extension APIs. A separate Captain View would be a new product layer, not a hidden current Codex feature.

▼ What survives if every visible session closes?

The briefs, backlogs, registries, metadata, status ledgers, queues, decisions, worktrees, branches, reports, and remote-home records remain on disk. The next valid lock-owning session uses those plus live backend facts to reconstruct reality.

Evidence boundary: this page is a plain-English map, not a second owner of script formats or exact command syntax. Follow the linked script headers, current help, tests, and authoritative documents when operating or extending Firstmate. See the evidence note for the principal sources and validation performed.